SRE - Assignment 1

Technical Debt Inventory and Analysis

of the Python-Markdown code base

Group Report

#	Group member	Roll number	Individual files (Appendix A)
1	Taha Yasir	i222429	`extensions/def_list.py`, `extensions/meta.py`
2	Khizar	i222595	`extensions/admonition.py`, `extensions/wikilinks.py`
3	Hadeed Abbas	i228780	`postprocessors.py`, `extensions/legacy_attrs.py`

Report, analysis scripts and raw tool outputs: https://github.com/Taha9876/SRE-A1

Contents

1. Project Overview 3
   1.1 Repository and version analysed 3
   1.2 System description and purpose 3
   1.3 Size and module count 3
2. Inventory Analysis 3
   2.1 Project assets 3
   2.2 Classification of assets 4
   2.3 Quality metrics per module 4
   2.4 Dependency mapping 5
3. Tool-Based Metrics 6
   3.1 Cyclomatic complexity 6
   3.2 Cognitive complexity 7
   3.3 Code duplication 7
   3.4 Test coverage 7
   3.5 Bugs, vulnerabilities and code smells 8
   3.6 Maintainability index 8
4. Debt Identification 8
   4.1 Example of code debt: a deeply nested method (TD-01) 9
   4.2 Example of design debt: one fix, two copies (TD-03) 9
   4.3 Example of test debt: tests tied to one library release (TD-04) 9
   4.4 Example of documentation debt (TD-05) 9
   4.5 Example of infrastructure debt (TD-06) 10
5. Remediation Cost and Debt Ratio 10
   5.1 Method 10
   5.2 Tool outputs and cost per item 10
   5.3 Technical Debt Ratio 10
6. Prioritization Strategy 11
   6.1 Technique 1: hotspot analysis 11
   6.2 Technique 2: weighted scoring model 12
   6.3 Prioritised order and justification 12
7. Reflection 12
Appendix A. Individual Component: Manual Calculations 14
   A.1 Member 1: Taha Yasir (i222429) 14
   A.2 Member 2: Khizar (i222595) 18
   A.3 Member 3: Hadeed Abbas (i228780) 22
Appendix B. Reproducing the Measurements 26

1. Project Overview

1.1 Repository and version analysed

Item	Value
Project	Python-Markdown (PyPI package `Markdown`)
Repository URL	https://github.com/Python-Markdown/markdown
Commit hash	`0bf535bc406c95dad66050f07c8402af03fe07d5`
Commit position	9 October 2026, branch `master`, 7 commits after release 3.11.0
Language / licence	Python (`requires-python >= 3.11`), BSD 3-Clause
History	Since March 2006; 1,672 commits, 191 contributors, 44 commits in the last 12 months
Our analysis repository	https://github.com/Taha9876/SRE-A1

1.2 System description and purpose

Python-Markdown converts Markdown text to HTML. It is used as a library, as the `markdown_py` command, and as the rendering engine of documentation tools such as MkDocs.

It is a five-stage pipeline run by the `Markdown` class: **preprocessors** clean the lines, the **block parser** builds an element tree, **tree processors** apply the **inline patterns**, a **serializer** writes HTML, and **postprocessors** restore stashed fragments. An extension API lets plug-ins add processors to any stage; 18 extensions ship with the package.

1.3 Size and module count

Business logic is the installable package `markdown/`. Tests, documentation and build files are not counted.

Measure (radon raw)	Core (14 modules)	Extensions (19 modules)	Package total
Physical lines of code (LOC)	4,651	3,958	8,609
Source lines (SLOC: no blanks, comments, docstrings)	2,279	2,172	4,451
Functions and methods	207	190	397
Classes	63	48	111

Selection criterion	Requirement	Python-Markdown	Met
Size	5,000 - 12,000 LOC	8,609 LOC (4,451 SLOC)	Yes
Structure	At least 10 classes / modules	33 modules, 111 classes	Yes
Complexity	Non-trivial decision logic	23 functions with cyclomatic complexity above 10; maximum 33	Yes
Language	Java, Python, C#, C++ or React	Python 3	Yes

2. Inventory Analysis

2.1 Project assets

Asset type	Location	Size	Content
Code - core	`markdown/*.py`	14 modules, 4,651 LOC	Pipeline engine, HTML parser, serializers, `util`, CLI
Code - extensions	`markdown/extensions/`	19 modules, 3,958 LOC	`Extension` base class and 18 plug-ins

Asset type	Location	Size	Content
Tests	`tests/`	37 files, 14,667 LOC, 1,123 test cases	`unittest` suites plus 300 fixture files
Libraries - runtime	`pyproject.toml`	0 mandatory, 2 optional	Standard library only; optional Pygments and PyYAML
Libraries - development	`pyproject.toml`, `tox.ini`	3 test, 8 documentation	`coverage`, `pyyaml`, `pytidylib`; pinned docs tool chain
Configuration / build	repository root	11 files	`pyproject.toml`, `tox.ini`, `makefile`, `.coveragerc`, `mkdocs.yml`, `uv.lock` and others
CI / process	`.github/`	6 workflows	Test matrix, changelog checks, docs deploy
Documentation	`docs/`, root `*.md`	68 Markdown pages	User guide, extension reference, API guide, changelog
UI	`markdown/__main__.py`	1 CLI module, 154 LOC	No graphical UI; command line `markdown_py` only

2.2 Classification of assets

Asset	Class	Evidence
`core.py`, `__init__.py`, `__meta__.py`, `__main__.py`	Active	Entry points and orchestrator
`blockparser.py`, `blockprocessors.py`, `inlinepatterns.py`, `treeprocessors.py`, `preprocessors.py`, `htmlparser.py`	Active	Default conversion path; touched by 31 of the 71 commits of the last three years
`postprocessors.py`	Active (part deprecated)	`UnescapePostprocessor` is marked `@deprecated`
`util.py`	Reusable	Generic `Registry` and helpers; imported by 16 modules
`extensions/__init__.py`	Reusable	`Extension` base class; imported by 20 modules
`serializers.py`	Reusable	Generic ElementTree-to-HTML writer with no internal dependency
`test_tools.py`	Reusable	`TestCase` helpers published for third-party extension authors
16 extensions (`abbr`, `admonition`, `attr_list`, `codehilite`, `def_list`, `extra`, `fenced_code`, `footnotes`, `md_in_html`, `meta`, `nl2br`, `sane_lists`, `smarty`, `tables`, `toc`, `wikilinks`)	Active	Registered entry points; documented and tested
`extensions/legacy_attrs.py`, `extensions/legacy_em.py`	Deprecated (legacy)	Kept only for documents written before version 3.0
`AbbrPreprocessor`, `AbbrInlineProcessor`, `toc.get_name`, `toc.stashedHTML2text`, `attr_list.get_attrs`, `Markdown.docType`	Deprecated	`@deprecated` decorator or a TODO marking it unused
`tests/test_legacy.py` and its 300 fixture files	Deprecated (legacy)	Older file-comparison tests
Other tests, documentation, CI, build configuration	Active	Maintained with each release

2.3 Quality metrics per module

CC = cyclomatic (radon); Cog = cognitive (complexipy); MI = maintainability index; Dup = duplicated lines (jscpd); Churn = commits in the last three years.

Module	LOC	SLOC	Fn	CC sum	CC max	Cog max	MI	Stmt %	Branch %	Dup	Churn
`__init__.py`	48	5	0	0	0	0	100.0	100.0	100.0	0	0
`__main__.py`	154	103	2	8	5	6	70.6	100.0	100.0	0	2
`__meta__.py`	51	14	1	6	6	3	75.9	100.0	100.0	0	13
`blockparser.py`	160	34	7	12	5	10	50.2	100.0	90.0	0	1
`blockprocessors.py`	642	346	34	113	11	18	34.6	97.7	94.4	50	5
`core.py`	531	197	12	41	10	14	48.9	100.0	94.7	32	5

Module	LOC	SLOC	Fn	CC sum	CC max	Cog max	MI	Stmt %	Branch %	Dup	Churn
extensions/__init__.py	145	34	7	14	3	2	53.3	100.0	90.0	0	2
extensions/abbr.py	199	115	15	37	10	18	59.5	93.3	91.2	16	11
extensions/admonition.py	192	98	7	29	13	19	62.4	100.0	100.0	23	4
extensions/attr_list.py	213	116	12	51	32	102	58.5	100.0	93.5	9	6
extensions/codehilite.py	355	178	9	40	14	28	58.9	99.1	92.9	7	4
extensions/def_list.py	128	63	5	21	17	19	69.9	96.6	100.0	12	3
extensions/extra.py	66	18	3	3	1	0	100.0	100.0	100.0	0	0
extensions/fenced_code.py	201	133	7	35	24	65	60.0	99.0	95.2	7	4
extensions/footnotes.py	494	312	33	80	7	19	40.3	97.3	84.8	22	6
extensions/legacy_attrs.py	78	27	5	11	7	9	79.3	100.0	100.0	12	2
extensions/legacy_em.py	59	20	2	2	1	0	89.2	100.0	100.0	34	1
extensions/md_in_html.py	458	291	22	123	25	94	32.9	98.0	92.0	38	9
extensions/meta.py	93	47	4	13	10	17	78.4	100.0	90.0	7	4
extensions/nl2br.py	49	10	2	2	1	0	100.0	100.0	100.0	7	2
extensions/sane_lists.py	77	26	4	4	1	0	83.2	100.0	100.0	7	3
extensions/smarty.py	295	151	10	17	5	4	65.8	100.0	78.6	8	6
extensions/tables.py	255	155	10	53	14	26	51.6	100.0	98.6	28	4
extensions/toc.py	496	323	26	98	14	25	37.0	94.5	91.7	11	5
extensions/wikilinks.py	105	55	7	13	5	7	78.7	100.0	100.0	11	3
htmlparser.py	460	290	28	101	14	19	36.5	99.0	90.0	9	15
inlinepatterns.py	1052	524	48	142	33	58	25.9	99.5	94.9	36	11
postprocessors.py	131	58	9	16	5	10	76.0	96.0	100.0	0	3
preprocessors.py	91	30	4	4	1	0	82.4	100.0	100.0	0	1
serializers.py	214	124	8	47	24	75	59.2	99.1	97.0	24	2
test_tools.py	224	93	9	21	7	28	68.4	91.5	94.4	0	0
treeprocessors.py	483	282	19	101	17	73	40.1	97.8	93.7	0	6
util.py	410	179	26	51	9	9	52.2	96.0	97.4	0	5
Package total / worst	**8,609**	**4,451**	**397**	**1,309**	**33**	**102**	**25.9**	**98.1**	**93.4**	**205**	**71**

2.4 Dependency mapping

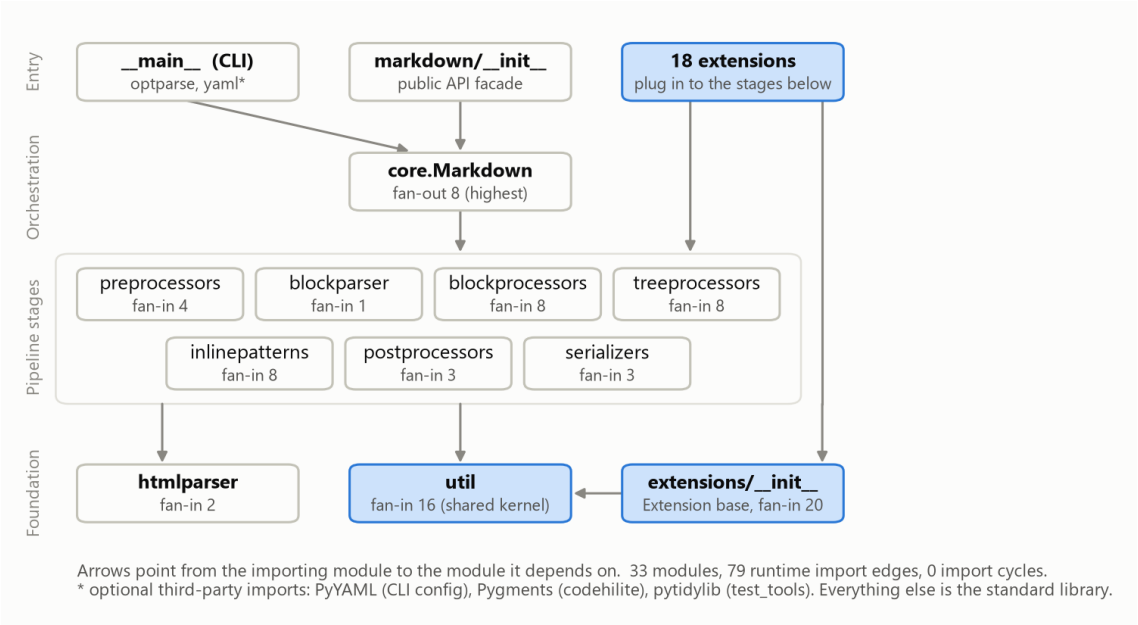


Figure 1. Layered view of the internal dependencies

Module	Fan-in (Ca)	Fan-out (Ce)	Instability Ce / (Ca + Ce)	Depends on
`extensions`	20	1	0.05	util
`util`	16	0	0.00	(nothing internal)
`blockprocessors`	8	2	0.20	blockparser, util
`inlinepatterns`	8	1	0.11	util
`treeprocessors`	8	2	0.20	inlinepatterns, util
`core`	1	8	0.89	blockprocessors, extensions, inlinepatterns, postprocessors, preprocessors, serializers, treeprocessors, util
`extensions.fenced_code`	0	6	1.00	extensions, extensions.attr_list, extensions.codehilite, preprocessors, serializers, util

- **No cycles:** 33 modules, 79 import edges, 0 import cycles.
- **Healthy direction:** stable modules (`util` 0.00, `Extension` base 0.05) are depended on; every extension (1.00) depends on them.
- **Concentration risk:** `util` has fan-in 16, so a change there reaches half of the code.
- **Hidden coupling:** `wikilinks` reads `Markdown.Meta`, an attribute only the `meta` extension creates.
- **External:** standard library only at run time; Pygments, PyYAML and pytidylib are optional.

3. Tool-Based Metrics

We could not keep a SonarQube server running on the analysis laptop, so we used the open-source tools below, as the assignment permits. Section 5 applies SonarQube's SQALE formula for the debt ratio.

Metric	Tool (version)	Result
Cyclomatic complexity	radon cc (6.0.1)	total 1,309; mean 3.30; maximum 33; 23 functions above 10
Cognitive complexity	complexipy (8.0.1)	total 1,743; mean 4.56; maximum 102; 26 functions above 15
Code duplication	jscpd (5.4.0)	2.38 % of lines (205 lines, 26 clone pairs)
Test coverage - statement	coverage.py (7.16.2)	98.07 % (3,514 of 3,583)
Test coverage - branch	coverage.py (7.16.2)	93.44 % (1,211 of 1,296)
Bugs	pylint (4.1.2)	0 errors; 20 reliability warnings
Vulnerabilities	bandit (1.9.4)	0 high, 0 medium, 25 low
Code smells	pylint, flake8 (7.4.1), vulture (2.16)	761 pylint messages; score 8.03 / 10
Maintainability index	radon mi (6.0.1)	mean 63.0; lowest 25.9; all modules rank A

3.1 Cyclomatic complexity

Most functions are simple: 336 of 397 (85 %) are rank A and the median is 2. The debt is in the tail: 23 functions exceed 10 and 9 exceed 15.

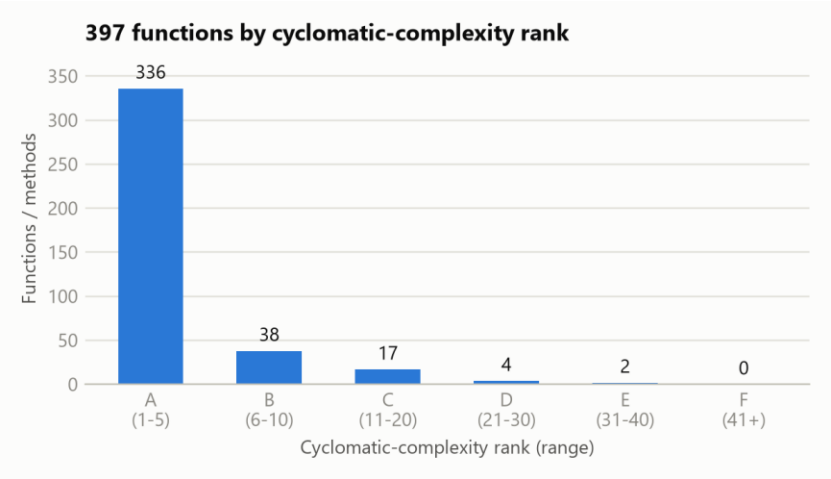


Figure 2. Distribution of cyclomatic complexity (radon ranks)

3.2 Cognitive complexity

26 functions exceed SonarQube's limit of 15 and 14 exceed 25. The ten worst:

Function	Module	Lines	Cyclomatic	Cognitive
`AttrListTreeprocessor.run`	`extensions/attr_list.py`	103-161	32	102
`HTMLExtractorExtra.handle_endtag`	`extensions/md_in_html.py`	160-246	25	94
`_serialize_html`	`serializers.py`	119-192	24	75
`InlineProcessor.__processPlaceholders`	`treeprocessors.py`	172-251	12	73
`FencedBlockPreprocessor.run`	`extensions/fenced_code.py`	91-171	24	65
`LinkInlineProcessor.getLink`	`inlinepatterns.py`	772-886	33	58
`HTMLExtractorExtra.handle_starttag`	`extensions/md_in_html.py`	110-158	23	41
`MarkdownInHtmlProcessor.parse_element_content`	`extensions/md_in_html.py`	328-399	14	37
`InlineProcessor.__applyPattern`	`treeprocessors.py`	253-335	17	36
`InlineProcessor.run`	`treeprocessors.py`	348-419	16	35

`AttrListTreeprocessor.run` and `getLink` have nearly equal cyclomatic complexity (32 and 33), but the first is nested six levels deep, so its cognitive complexity is almost double.

3.3 Code duplication

2.38 % of lines are duplicated (205 lines in 26 clone pairs), below SonarQube's 3 % gate. The clones that matter:

Clone	Location A	Location B
HTML escaping of &, <, >	`serializers.py` 70-78, 85-92	`extensions/toc.py` 120-127; `util.py` 158-168
Workaround for CPython bug gh-77057	`htmlparser.py` 372-380	`extensions/md_in_html.py` 292-302
Split a block around a match	`blockprocessors.py` 594-606	`extensions/abbr.py` 167-179; `extensions/footnotes.py` 286-300
Attribute escaping	`serializers.py` 83-94	`serializers.py` 102-113

3.4 Test coverage

Statement coverage is **98.07 %** (69 of 3,583 statements missed) and branch coverage is **93.44 %** (85 of 1,296 missed), from the project's 1,123 tests.

This needed two combined runs. The code-highlighting tests skip unless Pygments is exactly 2.7.1; in a normal environment 111 tests are skipped and `codehilite.py` drops to 11 % (overall 94 %).

Module (lowest branch coverage)	Statement %	Branch %	Missed statements	Missed branches
extensions/smarty.py	100.0	78.6	0	3
extensions/footnotes.py	97.3	84.8	7	14
blockparser.py	100.0	90.0	0	1
extensions/__init__.py	100.0	90.0	0	1
extensions/meta.py	100.0	90.0	0	1

3.5 Bugs, vulnerabilities and code smells

Category	Tool and rule	Count	Assessment
Bugs (confirmed)	pylint error / fatal	0	None provable statically
Bugs (potential)	super-init-not-called	8	Base initialiser skipped
	arguments-differ, arguments-renamed	6	Override changes the base signature
	inconsistent-return-statements	5	Value on some paths, None on others
	deprecated-method	1	logger.warn() at blockprocessors.py:490
Vulnerabilities	bandit B405 (xml.etree import)	20	False positives: no XML input is parsed
	bandit B101 (assert)	5	Low: removed under python -O
Code smells	pylint convention	554	line-too-long 196, invalid-name 152, missing docstrings 62
	pylint refactor	93	no-else-return 24, too-many-branches 12
	pylint warning (remaining)	98	attribute-defined-outside-init 36, redefined-builtin 29
	flake8	4	But 17 modules switch flake8 off file-wide (Section 4.4)
	vulture (dead code)	1	One unused variable

3.6 Maintainability index

All 33 modules are rank A (above 19) with a mean of 63.0, but the spread is wide: seven modules holding 47 % of the lines score below 41.

Module (lowest MI)	MI	LOC	Functions	CC sum
inlinepatterns.py	25.9	1052	48	142
extensions/md_in_html.py	32.9	458	22	123
blockprocessors.py	34.6	642	34	113
htmlparser.py	36.5	460	28	101
extensions/toc.py	37.0	496	26	98

4. Debt Identification

ID	Debt type	Weakness	Evidence
TD-01	Code debt	Over-complex functions	26 functions above cognitive 15; worst AttrListTreeprocessor.run (CC 32, cognitive 102)
TD-02	Design debt	Oversized module	inlinepatterns.py: 1,052 lines, 24 classes, lowest MI (25.9)
TD-03	Design debt	Duplicated logic	26 clone pairs; escaping helper exists three times
TD-04	Test debt	Pinned tests, branch gaps	111 tests skip unless Pygments is 2.7.1; 85 uncovered branches; 88 pragma: no cover
TD-05	Documentation debt	Undocumented or misleading code	62 missing docstrings (17 in htmlparser.py); stale TODOs
TD-06	Infrastructure debt	Stale version workarounds	Workaround "until we drop Python < 3.13" in two files; CLI on optparse

ID	Debt type	Weakness	Evidence
TD-07	Design debt (obsolescence)	Deprecated and dead code	10 items still shipped
TD-08	Code debt	Inconsistent naming	152 `invalid-name`: camelCase beside snake_case
TD-09	Code debt	Lint findings, suppressed checks	281 other pylint findings; 17 modules exempt from flake8
TD-10	Security debt	Assertions as run-time checks	5 `assert` statements in production paths

4.1 Example of code debt: a deeply nested method (TD-01)

`AttrListTreeprocessor.run` (`extensions/attr_list.py`, lines 103-161) repeats the same block three times at nesting depth five:

```
119    if pos is None and elem[-1].tail:
121        m = RE.search(elem[-1].tail)
122        if m:
123            if not self.assign_attrs(elem, m.group(1), strict=True):
124                elem[-1].tail = elem[-1].tail[:m.start()]
125    elif pos is not None and pos > 0 and elem[pos-1].tail:
127        m = RE.search(elem[pos-1].tail)
128        if m:
129            if not self.assign_attrs(elem, m.group(1), strict=True):
130                elem[pos-1].tail = elem[pos-1].tail[:m.start()]
131    elif elem.text:  ...  # the same block a third time
```

Result: cyclomatic 32, cognitive 102 (almost seven times the limit), four branches untested.

4.2 Example of design debt: one fix, two copies (TD-03)

The same workaround is in `htmlparser.py` (line 372) and `extensions/md_in_html.py` (line 292):

```
if self.rawdata[i:i+3] == '<![' and not self.rawdata[i:i+9] == '<![CDATA[':
    # We have encountered the bug in #1534 (Python bug `gh-77057`).
    # Provide an override until we drop support for Python < 3.13.
    result = self.parse_bogus_comment(i)
```

Both are among the five most-changed files, and 11 of the 15 commits to `htmlparser.py` were bug fixes, so every parser fix has to be made twice.

4.3 Example of test debt: tests tied to one library release (TD-04)

```
# tests/test_syntax/extensions/test_code_hilite.py
def setUp(self):
    if has_pygments and pygments.__version__ != required_pygments_version:
        self.skipTest(f'Pygments=={required_pygments_version} is required')
```

The expected HTML was captured from Pygments 2.7.1 (2020). With any other version the suite still passes while 111 tests are silently skipped, so a regression from a newer Pygments goes unnoticed.

4.4 Example of documentation debt (TD-05)

- `htmlparser.py`, the most-changed file: 18 of its 28 functions have no docstring.
- `core.py:119`: `# TODO: Maybe delete this. It does not appear to be used anymore.` on a public attribute.
- 17 modules contain `# flake8: noqa: E501 116-119`. It reads as an exemption for four lines, but we tested it and flake8 then reports nothing for the whole file.

4.5 Example of infrastructure debt (TD-06)

`htmlparser.py:169` still says "TODO: remove this when the bug is fixed in all supported Python versions", and the override in 4.2 waits for Python 3.13. Nothing links these workarounds to the supported-version setting, so they outlive their purpose.

5. Remediation Cost and Debt Ratio

5.1 Method

We applied the SQALE method used by SonarQube. The effort per finding is our own calibration; overlapping findings are costed once.

```
Remediation cost     = SUM over findings ( effort to fix the finding )
Development cost     = SLOC x 30 minutes          (SonarQube default: 30 min per line of code)
Technical Debt Ratio = Remediation cost / Development cost x 100 %
```

5.2 Tool outputs and cost per item

#	Finding (tool output)	Debt type	Count	Effort rule	Minutes
R1	Functions with cognitive complexity above 15	Code debt	26 functions	5 min + 1 min per point above 15	703
R2	Duplicated blocks	Design debt	26 clone pairs	20 min per clone pair	520
R3	Uncovered statements	Test debt	69 lines	2 min per line	138
R4	Uncovered branches	Test debt	85 branches	5 min per branch	425
R5	Tests pinned to Pygments 2.7.1	Test debt	1 test design issue	fixed estimate, 4 h	240
R6	Missing docstrings	Documentation debt	62 definitions	5 min each	310
R7	Non-PEP 8 names in the public API	Code debt	85 names	15 min each (alias + shim)	1,275
R8	Non-PEP 8 local names	Code debt	67 names	2 min each	134
R9	Other pylint findings	Code debt	281 findings	1-20 min each, by rule	988
R10	Lines over 119 columns	Code debt	62 lines	1 min each	62
R11	Security findings	Security debt	25 findings	10 min per assert, 2 min per review	90
R12	Oversized `inlinepatterns.py`	Design debt	1 module	fixed estimate, 8 h	480
R13	Deprecated or dead code	Design debt	10 items	15 min each	150
R14	Stale Python-version workarounds	Infrastructure debt	3 workarounds	30 min each	90
	Total remediation cost			**93.4 h = 11.7 person-days**	**5,605**

Example (R1): `handle_endtag` has cognitive complexity 94, so 5 + (94 - 15) = 84 minutes.

Debt type	Minutes	Hours	Share
Code debt	3,162	52.7	56 %
Design debt	1,150	19.2	21 %
Test debt	803	13.4	14 %
Documentation debt	310	5.2	6 %
Security debt	90	1.5	2 %
Infrastructure debt	90	1.5	2 %

5.3 Technical Debt Ratio

```
Remediation cost = 5,605 min  (93.4 hours)
Development cost = 4,451 SLOC x 30 min = 133,530 min  (2,225.5 hours)
TDR              = 5,605 / 133,530 x 100 = 4.20 %
```

SQALE maintainability rating	A	B	C	D	E
Technical Debt Ratio	up to 5 %	6 - 10 %	11 - 20 %	21 - 50 %	above 50 %
Python-Markdown	4.2 %				

Justification. 4.2 % is rating A, consistent with 97 % coverage, 2.4 % duplication and no import cycles. Three qualifications:

- **Size base:** we used SLOC as SonarQube does; with all 8,609 physical lines the ratio would be 2.2 %.
- **Concentration:** five of the 33 modules hold 61 % of the over-threshold cognitive complexity.
- **Not all should be repaid:** R7 (23 % of the total) means renaming public methods, which would break third-party extensions.

6. Prioritization Strategy

6.1 Technique 1: hotspot analysis

A hotspot is code that is both complex and often changed. Score = commits touching the module in the last three years (`git log`, 71 commits) x its cognitive complexity.

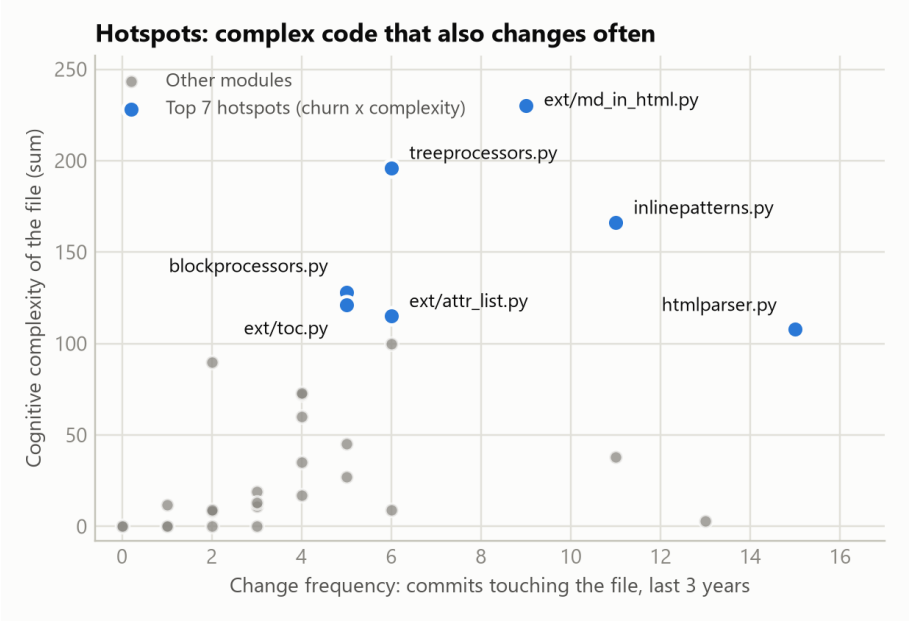


Figure 3. Change frequency against cognitive complexity per module

Rank	Module	Commits	Fixes	Cognitive	Score	Worst function (cognitive)
1	`extensions/md_in_html.py`	9	2	230	2,070	`HTMLExtractorExtra.handle_endtag (94)`
2	`inlinepatterns.py`	11	3	166	1,826	`LinkInlineProcessor.getLink (58)`
3	`htmlparser.py`	15	11	108	1,620	`HTMLExtractor.parse_starttag (19)`
4	`treeprocessors.py`	6	4	196	1,176	`InlineProcessor.__processPlaceholders (73)`
5	`extensions/attr_list.py`	6	1	115	690	`AttrListTreeprocessor.run (102)`
6	`blockprocessors.py`	5	2	128	640	`ParagraphProcessor.run (18)`

The top four modules hold 40 % of the cognitive complexity and were touched by 32 of the 71 commits. In `htmlparser.py`, 11 of 15 commits were fixes. `_serialize_html` is the third most complex function but its file changed only twice, so it is not a priority.

6.2 Technique 2: weighted scoring model

Each debt item is scored 1 to 5 on four criteria: **Impact** on users (weight 0.30), **Risk** of defects (0.25), **Change** frequency of the affected code (0.25) and **Ease** of repayment (0.20).

Rank	Item	Debt	Impact	Risk	Change	Ease	Score	Evidence
1	TD-01	Over-complex functions	5	5	5	2	4.40	Worst functions are in the four hotspots
2	TD-03	Duplicated logic	3	4	4	4	3.70	Parser fix duplicated in two hotspot files
3	TD-04	Pinned tests and branch gaps	4	4	3	3	3.55	111 skipped tests hide regressions
4	TD-02	Oversized `inlinepatterns.py`	3	3	5	2	3.30	11 commits in 3 years, MI 25.9
5	TD-06	Stale version workarounds	2	3	5	3	3.20	In `htmlparser.py`; partly blocked until 3.13
6	TD-05	Missing documentation	2	2	4	5	3.10	18 undocumented functions in `htmlparser.py`
7	TD-10	Assertions as run-time checks	3	2	2	5	2.90	5 statements, 50 minutes
8	TD-07	Deprecated and dead code	2	2	2	5	2.60	150 minutes
9	TD-09	Lint findings	1	2	3	4	2.35	Mostly automatic fixes
10	TD-08	Naming inconsistency	2	1	2	1	1.55	Renaming breaks third-party extensions

6.3 Prioritised order and justification

Both techniques put the same thing first: the top-scoring item (TD-01) lies in the top-ranked hotspot modules.

Order	Action	Items	Effort	Why now
1	Refactor the ten over-threshold functions in the four hotspot modules	TD-01 (part)	6 h	Top of both rankings
2	Merge the duplicated parser workaround and escaping helpers	TD-03 (part)	2 h	Cheap; prevents half-applied fixes
3	Unpin the Pygments tests; cover `smarty` and `footnotes` branches	TD-04	13 h	Safety net before step 5
4	Replace asserts, delete dead code, document `htmlparser.py`	TD-10, TD-07, TD-05	9 h	Quick wins
5	Split `inlinepatterns.py`; refactor the remaining complex functions	TD-02, TD-01 (rest)	14 h	Valuable but risky
6	Remove version workarounds when Python 3.13 is the minimum	TD-06	1.5 h	Blocked until then
7	Fix lint findings when a file is edited anyway	TD-09	17 h	No user impact
-	Do not rename the camelCase public API	TD-08	(21 h avoided)	Breaks third-party extensions

Steps 1 to 4 take about 30 hours, one third of the total remediation cost.

7. Reflection

Lessons learned

- **Averages hide debt.** Mean complexity 3.3 and 97 % coverage look healthy; the debt sits in 6 % of the functions and four files.
- **One metric is not enough.** Two functions with equal cyclomatic complexity differed twofold in cognitive complexity; coverage was 94 % or 97 % depending on one environment variable.
- **History is a metric.** Change frequency turned a list of complex functions into a ranking.

Trade-offs considered

- **Compatibility against cleanliness.** camelCase names and legacy extensions exist because third-party extensions depend on them, so we costed that debt but advise against repaying it.

- **Risk against benefit.** The parser needs refactoring most and is the most fix-prone, so test debt is ranked ahead of the large structural change.
- **Precision.** Our effort constants are estimates, so we rely on the ranking more than on the 4.2 % figure.

Implications for maintenance and evolution

- Manage debt where change happens: a CI rule against raising complexity in hotspot files costs almost nothing.
- Workarounds need an expiry: tie each to the minimum supported Python version in the release checklist.
- Suppressions (`noqa`, `pragma: no cover`) are debt too and should be reviewed like code.

Appendix A. Individual Component: Manual Calculations

Each member calculated the three metrics by hand for two files. The scanned handwritten sheets follow each member's summary. Rules used:

- **Cyclomatic complexity:** decision points + 1 per function (`if`, `elif`, `for`, `while`, `except`, each `and` / `or`, comprehension `for` / `if`, conditional expression).
- **Cognitive complexity** (SonarSource): +1 per `if`, `elif`, `else`, loop, `except`, conditional expression and boolean-operator sequence, plus +1 per nesting level for `if`, loops, `except` and conditional expressions.
- **Comment density** (SonarQube): comment lines / (comment lines + code lines) x 100; docstrings count as comments, blank lines are excluded.

A.1 Member 1: Taha Yasir (i222429)

File	Lines	Blank	# comment	Docstring	Code	Comment density
`extensions/def_list.py`	128	29	18	18	63	36 / (36 + 63) = 36.4 %
`extensions/meta.py`	93	22	11	13	47	24 / (24 + 47) = 33.8 %

File	Function	Cyclomatic	Cognitive
`extensions/def_list.py`	`DefListProcessor.run`	17	17
	`4 other functions with no decision point`	4 x 1	0
	`File total (hand) | tool cross-check`	21 | 21	17 | 19
`extensions/meta.py`	`MetaPreprocessor.run`	10	17
	`3 other functions with no decision point`	3 x 1	0
	`File total (hand) | tool cross-check`	13 | 13	17 | 17

Interpretation: extensions/def_list.py

Risks. One method holds the complexity: `DefListProcessor.run` has cyclomatic 17 (rank C); the other four functions have 1. It is driven by compound conditions (six `and` operators) and a `state` variable set in three places, and it returns `False` on one path and nothing on the others.

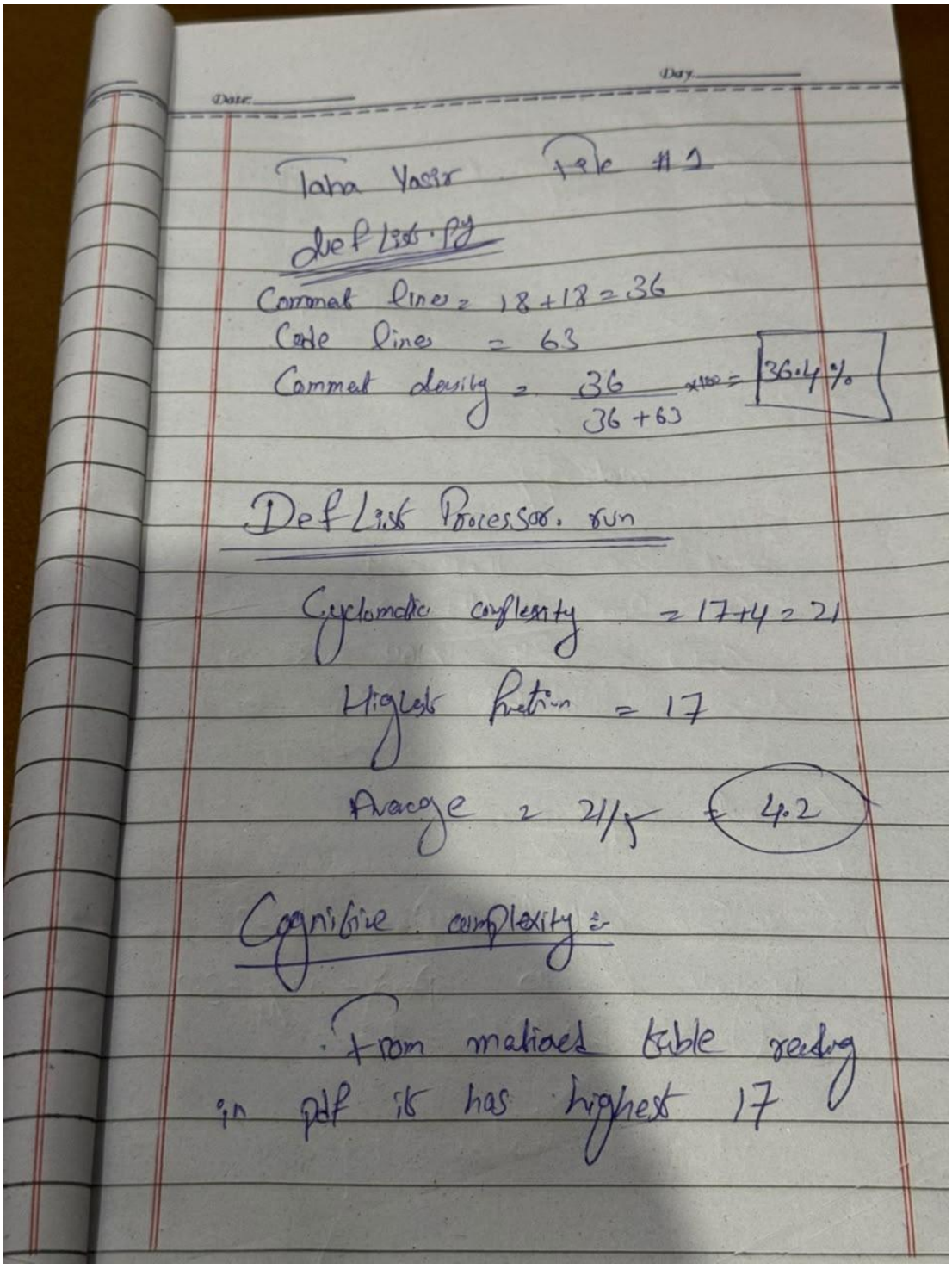
Debt and maintenance. Code debt with a documentation side: density is 36.4 %, but 15 of the 36 comment lines are licence header and a registration table, and `run` has no docstring. The file changed three times in three years, so interest is low; two extracted helpers would bring `run` under 10. The tool reports cognitive 19 against 17 by hand because it also counts the list comprehension on lines 43-44.

Interpretation: extensions/meta.py

Risks. `MetaPreprocessor.run` is a hand-written line parser: a `while` loop with two `break` statements, `try` / `except KeyError` used as normal control flow, and three levels of nesting. Cognitive complexity (17) is well above cyclomatic (10), so nesting is the problem. Branch coverage is 90 %: a document that ends inside the metadata block is untested.

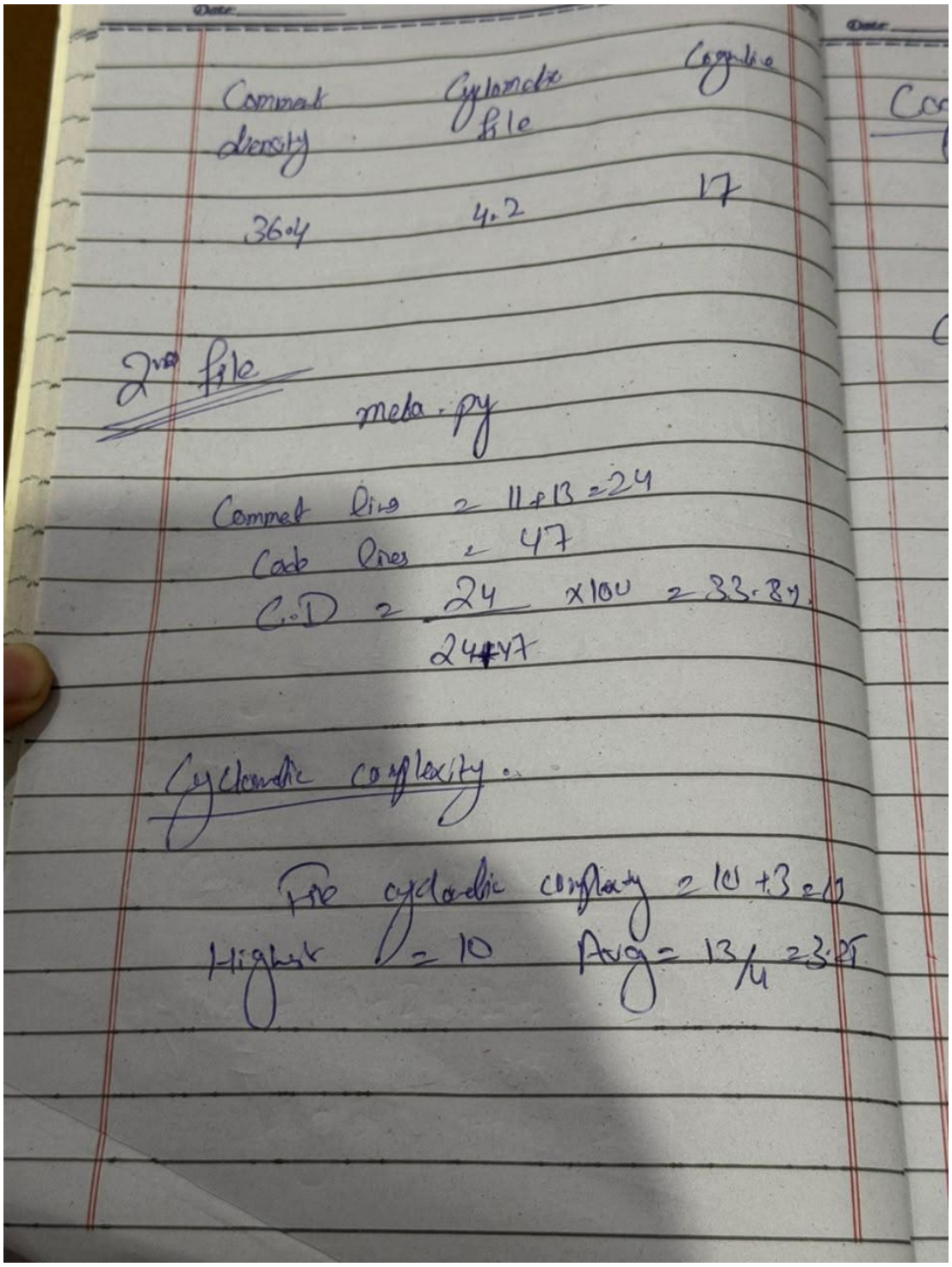
Debt and maintenance. Design debt: `MetaExtension.reset` creates `Markdown.Meta` on an object it does not own, and `wikilinks.py` reads it with `hasattr`. Two extensions are coupled through an attribute no class declares, so a change to the metadata format can break a reader the maintainer cannot find.

Handwritten calculations: Taha Yasir, sheet 1 of 3



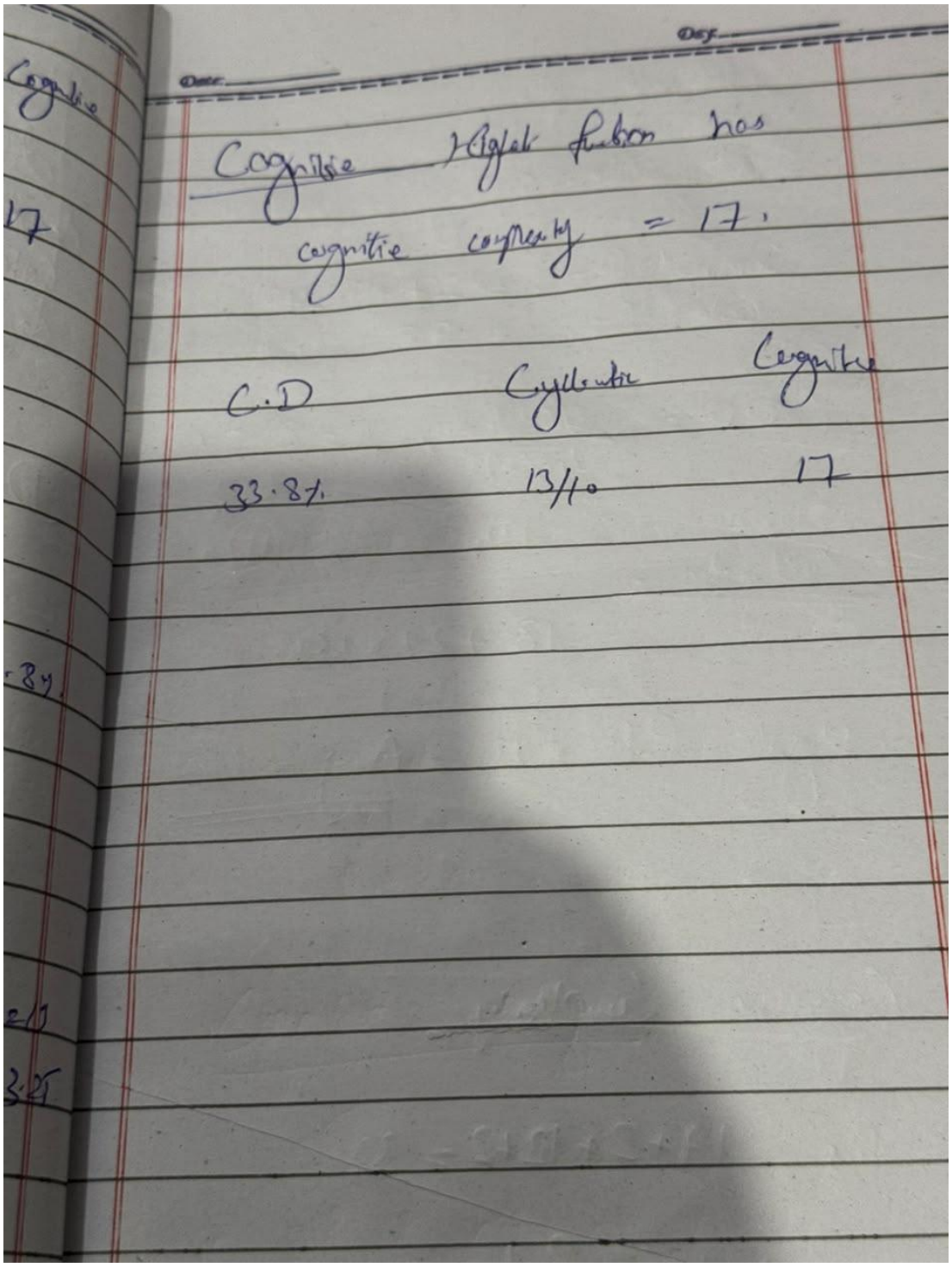
def_list.py: comment density, cyclomatic and cognitive complexity

Handwritten calculations: Taha Yasir, sheet 2 of 3



def_list.py summary; meta.py: comment density and cyclomatic complexity

Handwritten calculations: Taha Yasir, sheet 3 of 3



meta.py: cognitive complexity and summary

A.2 Member 2: Khizar (i222595)

File	Lines	Blank	# comment	Docstring	Code	Comment density
extensions/admonition.py	192	48	27	19	98	46 / (46 + 98) = 31.9 %
extensions/wikilinks.py	105	25	10	15	55	25 / (25 + 55) = 31.2 %

File	Function	Cyclomatic	Cognitive
extensions/admonition.py	AdmonitionProcessor.parse_content	13	19
	AdmonitionProcessor.test	2	2
	AdmonitionProcessor.run	8	12
	AdmonitionProcessor.get_class_and_title	3	2
	3 other functions with no decision point	3 x 1	0
	File total (hand) | tool cross-check	29 | 29	35 | 35
extensions/wikilinks.py	WikiLinksInlineProcessor.handleMatch	3	4
	WikiLinksInlineProcessor._getMeta	5	7
	5 other functions with no decision point	5 x 1	0
	File total (hand) | tool cross-check	13 | 13	11 | 11

Interpretation: extensions/admonition.py

Risks. `parse_content` (cyclomatic 13, cognitive 19) nests a four-part `and` and a conditional expression inside a `while` loop; the conditional expression alone costs four points at nesting level 3. `test()` stores `current_sibling` and `content_indent` for `run()`, so the two methods are only correct when called in that order.

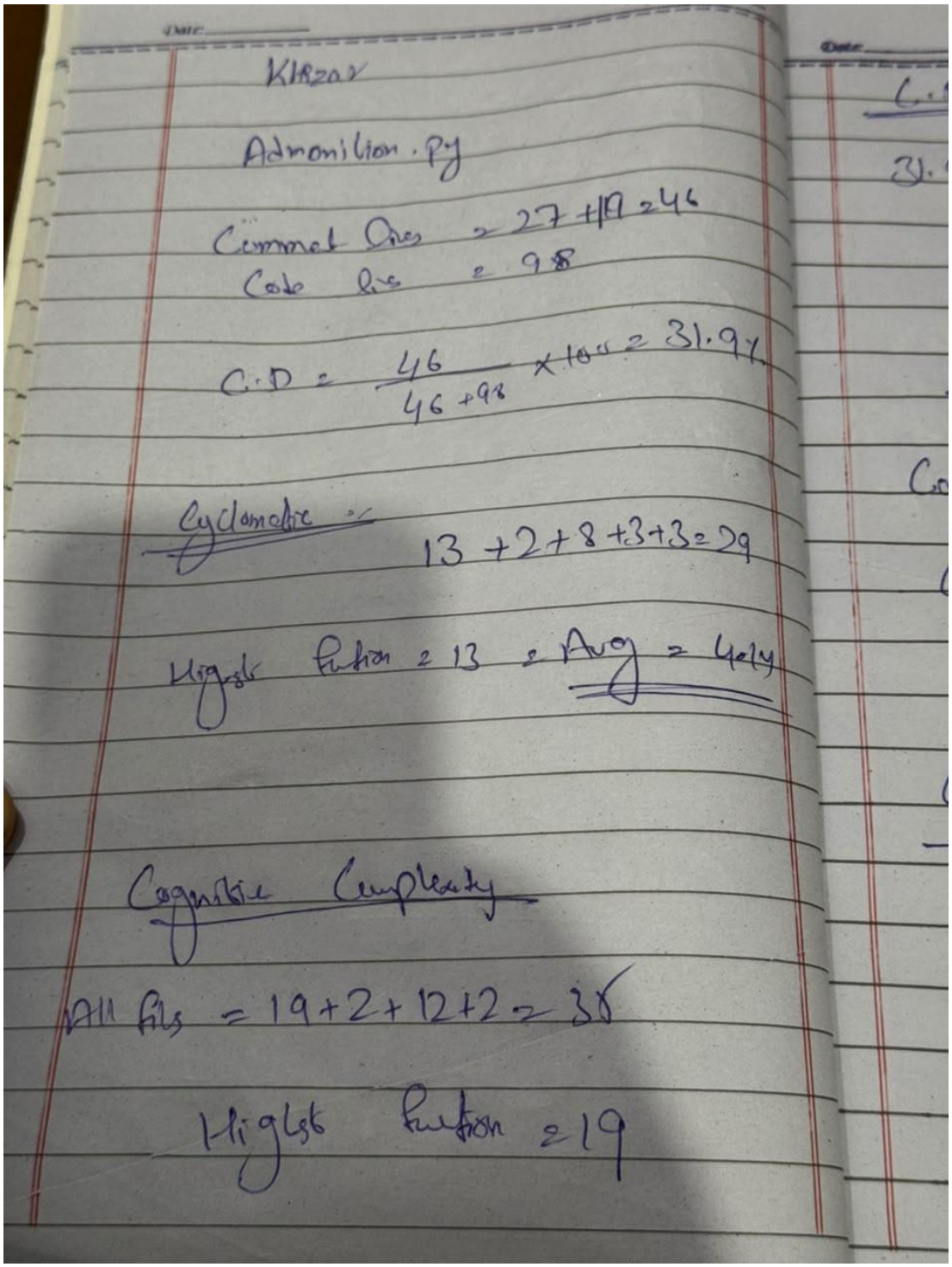
Debt and maintenance. Code debt plus design debt (a hidden calling protocol). The class and three methods have no docstring and the comment on line 159 is garbled, so the 31.9 % density overstates the documentation. Statement and branch coverage are both 100 %, which makes refactoring safe.

Interpretation: extensions/wikilinks.py

Risks. Low: every function is rank A and the highest cyclomatic value is 5. `_getMeta` repeats the same `if 'wiki_...' in self.md.Meta` block three times, and `handleMatch` returns an `Element` on one path and an empty string on the other.

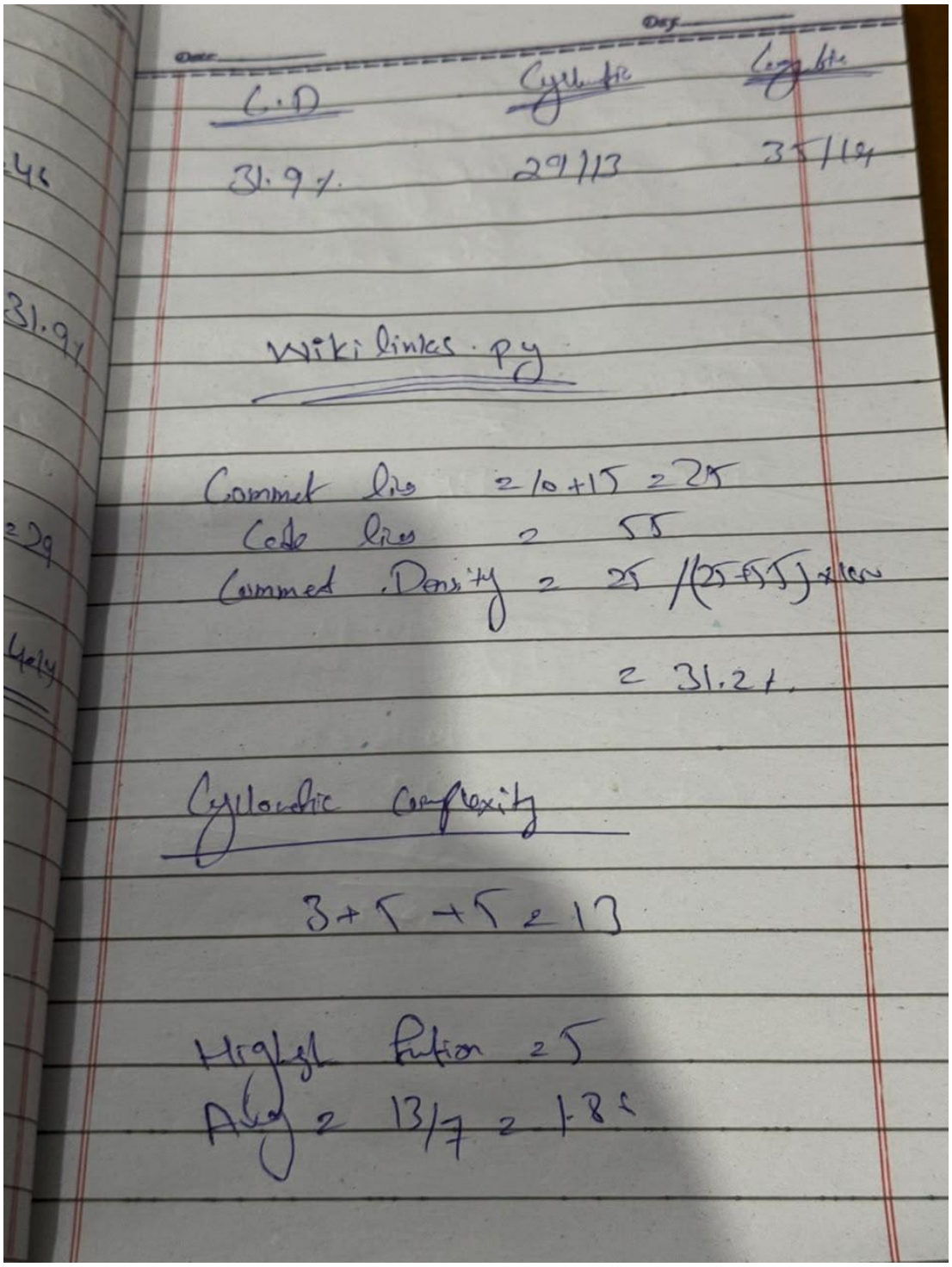
Debt and maintenance. Mild design debt: the file depends on the `meta` extension through the undeclared `Markdown.Meta` attribute. Fixing everything would take about half an hour and the file changed three times in three years, so it is low priority and a baseline for a low-debt extension.

Handwritten calculations: Khizar, sheet 1 of 3



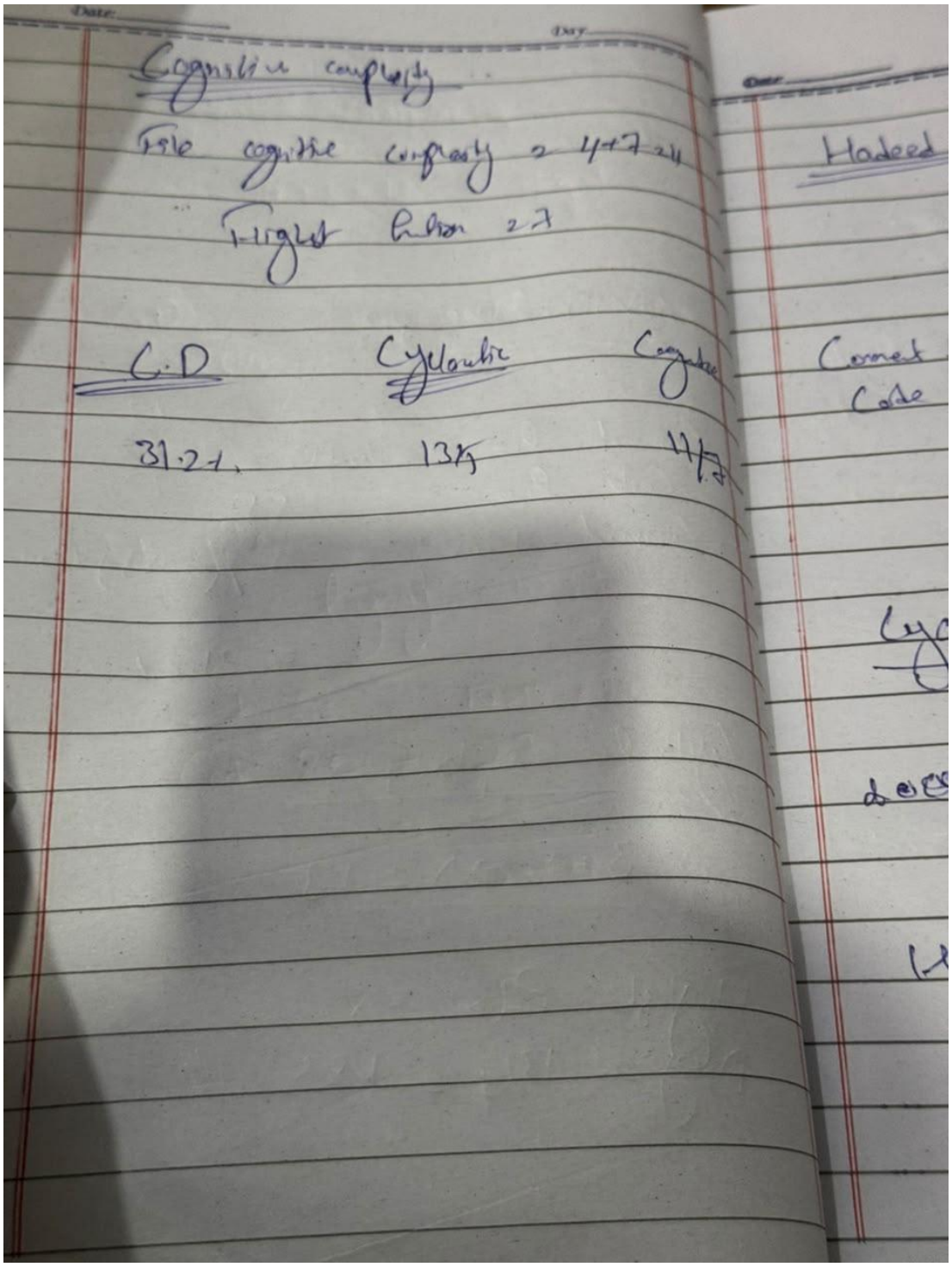
admonition.py: comment density, cyclomatic and cognitive complexity

Handwritten calculations: Khizar, sheet 2 of 3



admonition.py summary; wikilinks.py: comment density and cyclomatic complexity

Handwritten calculations: Khizar, sheet 3 of 3



wikilinks.py: cognitive complexity and summary

A.3 Member 3: Hadeed Abbas (i228780)

File	Lines	Blank	# comment	Docstring	Code	Comment density
postprocessors.py	131	35	14	24	58	38 / (38 + 58) = 39.6 %
extensions/legacy_attrs.py	78	21	14	16	27	30 / (30 + 27) = 52.6 %

File	Function	Cyclomatic	Cognitive
postprocessors.py	RawHtmlPostprocessor.run	2	10
	RawHtmlPostprocessor.run > substitute_match	5	(in parent)
	RawHtmlPostprocessor.isblocklevel	3	3
	6 other functions with no decision point	6 x 1	0
	File total (hand) | tool cross-check	16 | 16	13 | 13
extensions/legacy_attrs.py	LegacyAttrs.run	7	9
	4 other functions with no decision point	4 x 1	0
	File total (hand) | tool cross-check	11 | 11	9 | 9

Interpretation: postprocessors.py

Risks. `RawHtmlPostprocessor.run` hides its logic in a nested function that calls itself through `pattern.sub(substitute_match, html)`, so recursion depth depends on the input. Per-function tools understate it: radon reports 2 for `run` and 5 for the closure, while cognitive complexity charges the closure to its parent and gives 10.

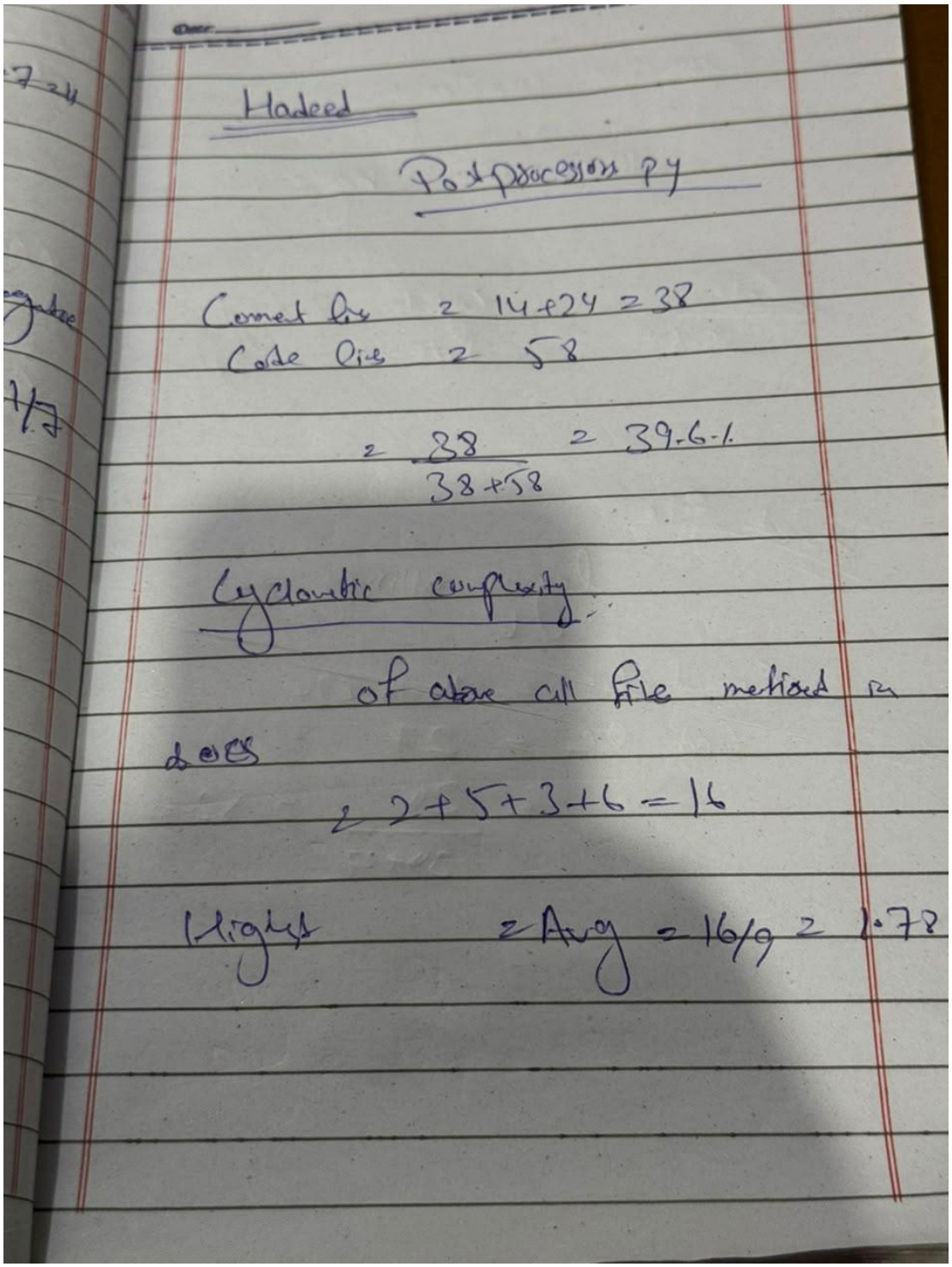
Debt and maintenance. Obsolescence debt: `UnescapePostprocessor` is `@deprecated` but still shipped, and its two methods are the file's only uncovered lines. Documentation debt: three docstrings contain errors (lines 22, 49, 96) and the base `Postprocessor.run` is a bare `pass`, so a subclass that forgets to override it silently returns `None`.

Interpretation: extensions/legacy_attrs.py

Risks. Only 27 code lines. `LegacyAttrs.run` reaches cyclomatic 7 from three `if` statements, two with an `and`, inside one loop. The callback in `handleAttributes` changes the element as a side effect and returns nothing, relying on `re.sub` treating `None` as an empty replacement.

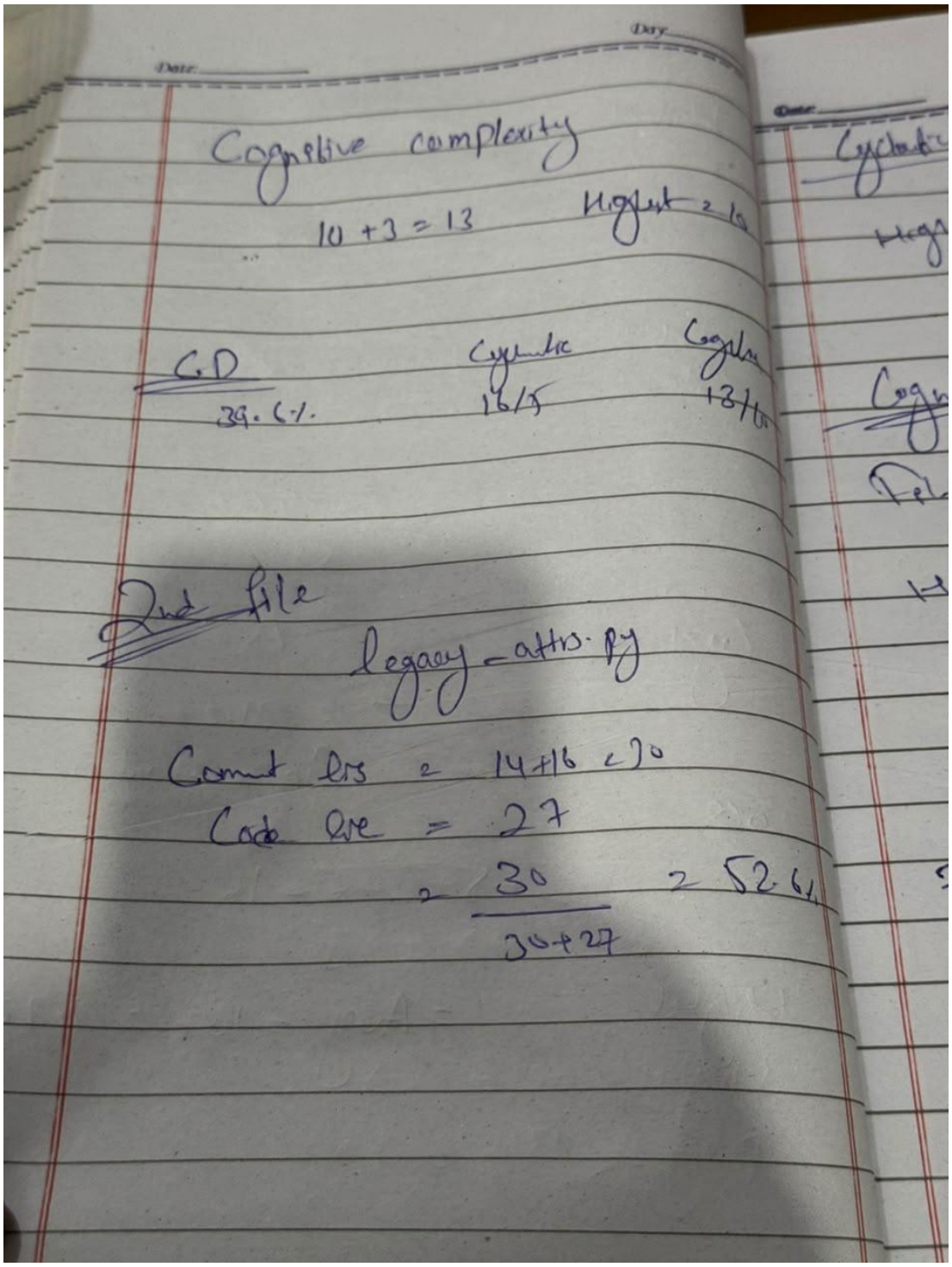
Debt and maintenance. The module is deliberate debt, kept so that pre-3.0 documents still render. Its 52.6 % comment density is the highest measured, but 13 of the 30 comment lines are licence header and neither class has a docstring. The cost is small but permanent: it must pass on every new Python version until a major release removes it.

Handwritten calculations: Hadeed Abbas, sheet 1 of 3



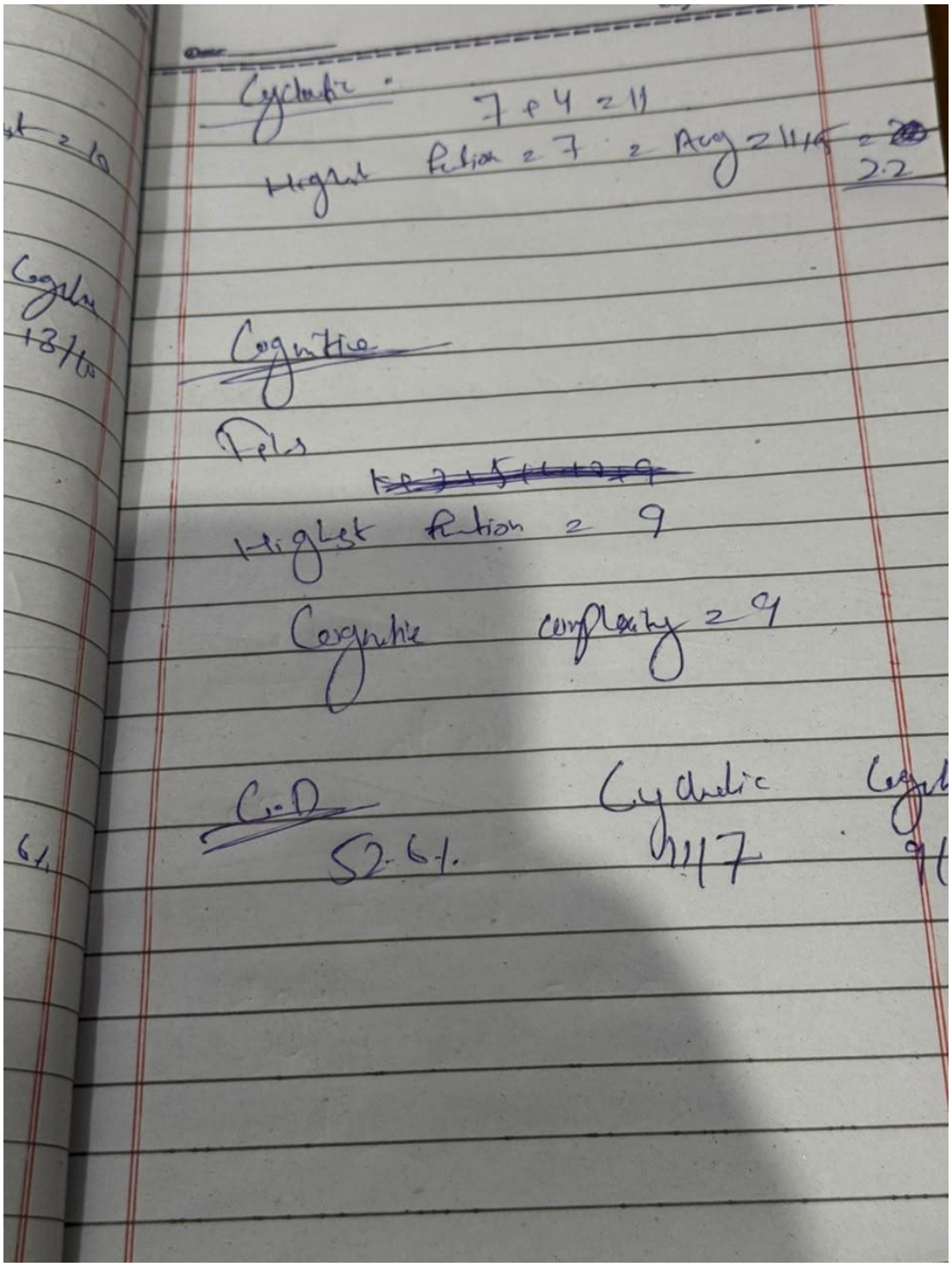
postprocessors.py: comment density and cyclomatic complexity

Handwritten calculations: Hadeed Abbas, sheet 2 of 3



postprocessors.py cognitive complexity and summary; legacy_attrs.py: comment density

Handwritten calculations: Hadeed Abbas, sheet 3 of 3



legacy_attrs.py: cyclomatic and cognitive complexity, summary

Appendix B. Reproducing the Measurements

Scripts and raw tool outputs: https://github.com/Taha9876/SRE-A1

```
git clone https://github.com/Python-Markdown/markdown.git && cd markdown
git checkout 0bf535bc406c95dad66050f07c8402af03fe07d5

radon raw markdown -j        # size, comment lines
radon cc  markdown -s -a -j  # cyclomatic complexity
radon mi  markdown -s -j     # maintainability index
complexipy markdown --output-format json           # cognitive complexity
npx jscpd markdown --min-lines 5 --min-tokens 50    # duplication
pylint markdown --output-format=json2 --exit-zero   # bugs / code smells
bandit -r markdown -f json                          # vulnerabilities
flake8 markdown --max-line-length=119 ; vulture markdown --min-confidence 80

# coverage: two runs, combined; .coveragerc set aside so test_tools.py is measured too
coverage run -p --branch --source=markdown -m unittest discover tests      # without Pygments
pip install pygments==2.7.1 && set PYGMENTS_VERSION=2.7.1
coverage run -p --branch --source=markdown -m unittest discover tests      # with pinned Pygments
coverage combine && coverage report

git log --since="3 years ago" --name-only --format=@%H^|%s -- markdown     # change frequency
```

References

- T. J. McCabe, "A Complexity Measure", IEEE Transactions on Software Engineering, 1976.
- G. A. Campbell, "Cognitive Complexity", SonarSource, 2018.
- J.-L. Letouzey, "The SQALE Method for Managing Technical Debt", 2016.
- A. Tornhill, "Your Code as a Crime Scene", Pragmatic Bookshelf, 2015.